BACKEND GELİŞTİRME
PROJE ENVANTERİ

TaskFlow API

Görev ve Proje Yönetim Sistemi — REST API

Bir yazılım ekibinin görevlerini, sorumlularını, durumlarını ve önceliklerini uçtan uca yöneten; Node.js ve Express ile katmanlı mimaride geliştirilmiş, tamamen JSON tabanlı bir REST API projesinin bileşen, endpoint, veri modeli ve test envanteri.



Node.js · Express 5.2 · REST · JSON · Katmanlı Mimari · Middleware · Dosya Tabanlı Veri Deposu · Postman

HAZIRLAYAN
İbrahim Gürkan Kaya

DERS
Node.js ile Backend Programlama — Bitirme Projesi

SÜRÜM
v1.0.0

TARİH
Ekim 2026

DOKÜMAN HARİTASI

İçindekiler

01 Proje Tanıtımı 3
02 Teknoloji Envanteri 3
03 Mimari Tasarım 4
04 Dosya ve Modül Envanteri 6
05 Veri Modeli Tasarımı 7
06 Endpoint Envanteri 8
07 Middleware Envanteri 9
08 Hata Yönetimi ve Durum Kodları 9
09 Örnek İstek ve Yanıtlar 10
10 Test Envanteri (Postman) 11
11 Gereksinim Karşılama Matrisi 12
12 Kurulum ve Çalıştırma 13
13 Gelecek Sürüm Planı 13

PROJE KİMLİĞİ

Proje Künyesi

PROJE ADI	TaskFlow — Görev ve Proje Yönetim Sistemi API
PAKET ADI	`taskflow-api` · sürüm `1.0.0`
PROJE TÜRÜ	Backend · REST API (arayüzsüz, JSON giriş/çıkış)
DERS	Node.js ile Backend Programlama — Bitirme Projesi
HAZIRLAYAN	İbrahim Gürkan Kaya
ÇALIŞMA ORTAMI	Node.js v18+ · Express 5.2.1 · varsayılan port `3000`
GİRİŞ NOKTASI	`src/server.js` → `src/app.js`
VERİ SAKLAMA	Dosya tabanlı JSON deposu — `src/data/tasks.json`
BASE URL	`http://localhost:3000`
TESLİM İÇERİĞİ	Kaynak kod, README, 4 proje dokümanı, Postman koleksiyonu, test ekran görüntüleri

ENVANTER ÖZETİ

Sayısal Görünüm

11
HTTP route
(10 API + 1 sağlık kontrolü)

10
Controller handler
(7 görev + 3 rapor)

5
Özel middleware fonksiyonu
(3 dosyada)

8
Task veri modeli alanı
(3 otomatik üretilir)

11
JavaScript kaynak dosyası
(`src/` altında)

359
Satır uygulama kodu
(yorumlar dahil)

16
Postman isteği
(3 klasör + sağlık kontrolü)

2 +1
Bağımlılık
(2 çalışma + 1 geliştirme)

AMAÇ · SENARYO · KAPSAM

01 Proje Tanıtımı

Bir yazılım şirketi; ekip içerisindeki görevleri, projeleri ve çalışanların sorumluluklarını takip etmek istemektedir. TaskFlow API, bir görevin yaşam döngüsünü — **oluşturma → atama → durum takibi → tamamlanma** — uçtan uca yöneten ve tamamen REST prensipleriyle tasarlanmış bir backend servisidir.

Görev Yönetimi

Görev ekleme, listeleme, detay görüntüleme, kısmi güncelleme ve silme (CRUD). Her görev bir çalışana (`assignee`) atanır.

Durum ve Öncelik

Durumlar: `pending` , `in-progress` , `completed` . Öncelikler: `low` , `medium` , `high` .

İzlenebilirlik

Gelen her istek method, endpoint ve zaman damgasıyla konsola ve `logs/requests.log` dosyasına yazılır.

Veri Doğrulama

Eksik veya hatalı alanlar, alan bazlı açıklayıcı mesajlarla `400 Bad Request` olarak reddedilir.

Sorgulama

Duruma/önceliğe göre filtreleme, anahtar kelimeyle arama, kişiye göre listeleme, sayfalama ve sıralama.

Raporlama

Tamamlanan ve bekleyen görev listeleri ile durum ve öncelik dağılımını veren genel özet raporu.

Kapsam içi

- REST API tasarımı ve Express routing yapısı
- CRUD operasyonları ve JSON tabanlı veri yönetimi
- Loglama, doğrulama ve hata yönetimi middleware'leri
- Filtreleme, arama, sayfalama, sıralama, raporlama
- Dokümantasyon ve Postman test koleksiyonu

Kapsam dışı

- Kullanıcı kimlik doğrulama ve yetkilendirme
- Harici / ilişkisel veritabanı entegrasyonu
- Kullanıcı arayüzü (frontend)

Sistem tamamen backend REST API odaklıdır; istemci olarak Postman, curl veya tarayıcı kullanılır.

ÇALIŞMA ZAMANI · KÜTÜPHANELER · ARAÇLAR

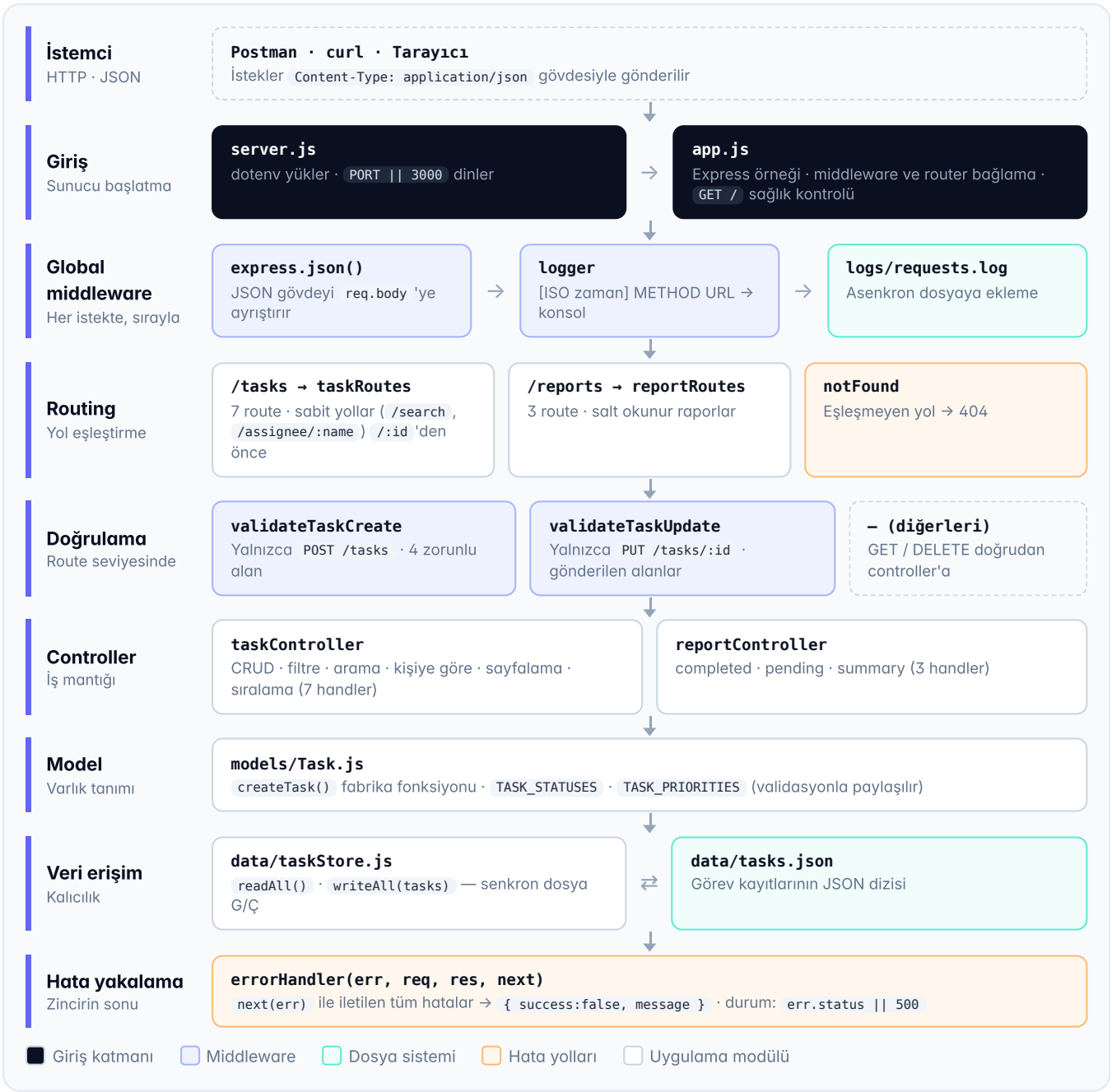
02 Teknoloji Envanteri

BİLEŞEN	SÜRÜM	TÜR	PROJEDEKİ ROLÜ
Node.js	`v18+`	Çalışma ortamı	Sunucu tarafı JavaScript çalışma zamanı
Express.js	`5.2.1`	Bağımlılık	HTTP sunucusu, routing, middleware zinciri, `express.json()` gövde ayrıştırıcı
dotenv	`18.0.3`	Bağımlılık	`.env` dosyasından `PORT` ortam değişkenini yükler
nodemon	`3.1.14`	Geliştirme	`npm run dev` — dosya değişikliklerinde otomatik yeniden başlatma
crypto (yerleşik)	–	Node çekirdek	`randomUUID()` ile benzersiz görev kimliği üretimi
fs · path (yerleşik)	–	Node çekirdek	JSON veri deposunun okunması/yazılması ve log dosyasına ekleme
Postman	`v2.1 şema`	Test aracı	16 hazır istek içeren koleksiyon; `baseUrl` ve `taskId` değişkenleri
Git	–	Sürüm kontrol	Kaynak kod ve doküman geçmişi; `node_modules` , `.env` , log dosyaları hariç tutulur

KATMANLAR · SORUMLULUKLAR · AKIŞ

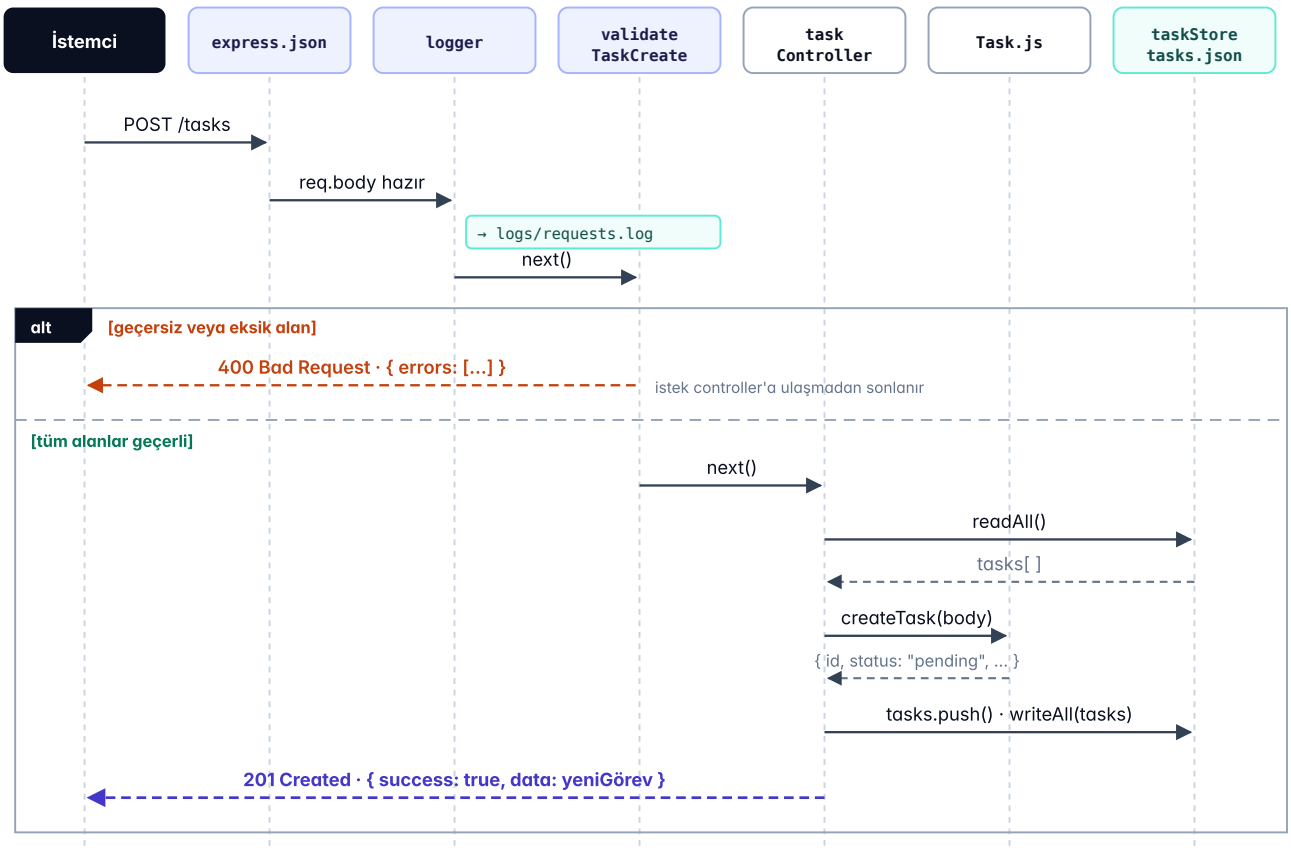
03 Mimari Tasarım

Uygulama, her katmanın tek bir sorumluluğa sahip olduğu **katmanlı (layered) bir mimariyle** kurgulanmıştır. Bir HTTP isteği yukarıdan aşağıya doğru middleware zincirinden geçer, ilgili router tarafından eşleştirilir, gerekiyorsa doğrulanır ve controller'a ulaşır; controller veri erişim katmanı üzerinden JSON deposuyla konuşur.



İstek yaşam döngüsü — `POST` `/tasks`

Aşağıdaki sıralama diyagramı, yeni görev oluşturma isteğinin katmanlar arasında nasıl ilerlediğini; geçersiz veri durumunda isteğin doğrulama katmanında nasıl sonlandırıldığını gösterir.



Tasarım kararları

Sorumlulukların ayrılması

Route tanımları, doğrulama, iş mantığı ve veri erişimi ayrı modüllerdedir. Controller'lar dosya sistemini doğrudan bilmez; yalnızca `taskStore` arayüzünü (`readAll` / `writeAll`) kullanır. Depolama değiştirildiğinde yalnızca bu modül güncellenir.

Tek kaynaktan enum'lar

`TASK_STATUSES` ve `TASK_PRIORITIES` sabitleri `models/Task.js` içinde tanımlanır ve doğrulama middleware'i tarafından içe aktarılır; izin verilen değerler tek yerde yönetilir.

Route sıralaması

`/tasks/search` ve `/tasks/assignee/:name` yolları, `/tasks/:id` 'den önce tanımlanmıştır. Böylece `"search"` ifadesi bir görev kimliği olarak yorumlanmaz.

Tutarlı yanıt sözleşmesi

Tüm yanıtlar `success` alanını içerir. Başarıda `data` (liste uçlarında `total` / `count`), hatada `message` ve gerekirse `errors[]` döner.

KLASÖR YAPISI · MODÜLLER · DIŞA AKTARIMLAR

04 Dosya ve Modül Envanteri

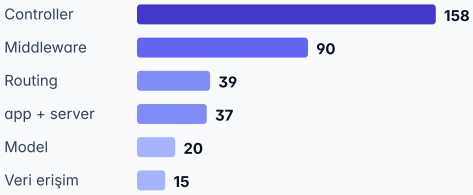
Uygulama kodu

```
TNC_Backend/
└ src/
   ├ controllers/
   │  ├ taskController.js
   │  └ reportController.js
   ├ data/
   │  ├ taskStore.js
   │  └ tasks.json
   ├ middleware/
   │  ├ logger.js
   │  ├ validateTask.js
   │  └ errorHandler.js
   ├ models/
   │  └ Task.js
   ├ routes/
   │  ├ taskRoutes.js
   │  └ reportRoutes.js
   ├ app.js
   └ server.js
```

Doküman, test ve yapılandırma

```
TNC_Backend/
├ docs/
│  ├ PROJE_TANITIMI.md
│  ├ VERI_MODELI.md
│  ├ API_TASARIM.md
│  ├ KULLANIM_KILAVUZU.md
│  └ screenshots/ (2 jpg)
├ postman/
│  └ TaskFlow.postman_
│     collection.json
├ logs/
│  └ requests.log
├ .env.example
├ .gitignore
├ package.json
├ package-lock.json
└ README.md
```

Kod dağılımı · 11 dosya, 359 satır



Her klasör tek bir katmana karşılık gelir; bağımlılık yönü tek yönlüdür: **route → controller → store**.

Kaynak modül envanteri

DOSYA (src/)	KATMAN	SORUMLULUK	DIŞA AKTARILANLAR	SATIR
`server.js`	Giriş	Ortam değişkenlerini yükler, sunucuyu `PORT` üzerinde başlatır	—	8
`app.js`	Uygulama	Express örneği, middleware sırası, router bağlama, sağlık kontrolü	`app`	29
`routes/taskRoutes.js`	Routing	`/tasks` altındaki 7 yol ve doğrulama bağlantıları	`router`	25
`routes/reportRoutes.js`	Routing	`/reports` altındaki 3 rapor yolu	`router`	14
`controllers/taskController.js`	Controller	CRUD, filtreleme, arama, kişiye göre listeleme, sayfalama, sıralama	`getAllTasks, searchTasks, getTasksByAssignee, getTaskById, createTaskHandler, updateTaskHandler, deleteTaskHandler`	126
`controllers/reportController.js`	Controller	Tamamlanan / bekleyen listeleri ve özet istatistikler	`getCompletedReport, getPendingReport, getSummaryReport`	32
`middleware/logger.js`	Middleware	İstek loglama (konsol + dosya), log klasörünü yoksa oluşturur	`logger`	23
`middleware/validateTask.js`	Middleware	Oluşturma ve güncelleme gövdelerinin doğrulanması	`validateTaskCreate, validateTaskUpdate`	54
`middleware/errorHandler.js`	Middleware	Tanımsız route (404) ve beklenmeyen hata (500) yanıtları	`notFound, errorHandler`	13
`models/Task.js`	Model	Görev nesnesi fabrikası, UUID ve zaman damgası üretimi, enum'lar	`createTask, TASK_STATUSES, TASK_PRIORITIES`	20
`data/taskStore.js`	Veri erişim	`tasks.json` dosyasının okunması ve yazılması	`readAll, writeAll`	15

VARLIK · ALANLAR · KURALLAR

05 Veri Modeli Tasarımı

Sistemdeki tek veri varlığı **Task** (Görev) nesnesidir. Görevler `src/data/tasks.json` dosyasında JSON dizisi olarak saklanır; her yazma işleminden sonra dosya güncellenir, böylece sunucu yeniden başlatıldığında veriler korunur.

ALAN · TİP	KAYNAK	KURAL / AÇIKLAMA
id string (UUID)	OTOMATİK	`crypto.randomUUID()` ile benzersiz kimlik
title string	ZORUNLU	Boş olmayan metin · görev başlığı
description string	ZORUNLU	Boş olmayan metin · görev açıklaması
status enum	OPSİYONEL	`pending` · `in-progress` · `completed` ; varsayılan `pending`
priority enum	ZORUNLU	`low` · `medium` · `high`
assignee string	ZORUNLU	Görevin atandığı çalışanın adı
createdAt ISO 8601	OTOMATİK	Oluşturulma anı; güncellemede korunur
updatedAt ISO 8601	OTOMATİK	Her `PUT` işleminde yenilenir

Örnek kayıt `src/data/tasks.json`

```
{
  "id": "385a8373-d33e-4fd8-9c01-805adf67a2fd",
  "title": "Backend API tasarla",
  "description": "TaskFlow REST API endpointlerini tasarla",
  "status": "pending",
  "priority": "high",
  "assignee": "musa",
  "createdAt": "2026-10-18T13:03:31.271Z",
  "updatedAt": "2026-10-18T13:03:31.271Z"
}
```

Sunucu, istemciden gelen `id` ve `createdAt` değerlerini dikkate almaz; bu alanlar yalnızca `models/Task.js` tarafından üretilir.

Görev durum akışı



Doğrulama kuralları

ALAN	OLUŞTURMA — validateTaskCreate	GÜNCELLEME — validateTaskUpdate
`title`	Zorunlu, string, boşluk dışı en az bir karakter	Gönderildiyse boş olamaz
`description`	Zorunlu, string, boş olamaz	Gönderildiyse boş olamaz
`priority`	Zorunlu; `low | medium | high`	Gönderildiyse `low | medium | high`
`assignee`	Zorunlu, string, boş olamaz	Gönderildiyse boş olamaz
`status`	Gönderilmezse `pending` atanır	Gönderildiyse `pending | in-progress | completed`
`id, createdAt`	Sunucu üretir	Gövdede gönderilse bile korunur, değiştirilemez

Tüm kural ihlalleri tek seferde toplanır ve `errors` dizisinde birlikte döndürülür; istemci tek istekte bütün eksikleri görür.

YOLLAR · HANDLER'LAR · YANIT KODLARI

06 Endpoint Envanteri

#	METHOD	ENDPOİNT	AÇIKLAMA	HANDLER · MİDDLEWARE	YANITLAR
GENEL					
0	GET	/	Sağlık kontrolü; API'nin ayakta olduğunu ve kök yolları döndürür	app.js (inline)	200
GÖREVLER — CRUD					ZORUNLU
1	POST	/tasks	Yeni görev oluşturur; `status` verilmezse `pending`	validateTaskCreate → createTaskHandler	201 400
2	GET	/tasks	Tüm görevleri listeler; filtre, sayfalama ve sıralama parametrelerini kabul eder	getAllTasks	200
3	GET	/tasks/:id	Tek görevin detayını getirir	getTaskById	200 404
4	PUT	/tasks/:id	Kısmi güncelleme; yalnızca gönderilen alanlar değişir, `updatedAt` yenilenir	validateTaskUpdate → updateTaskHandler	200 400 404
5	DELETE	/tasks/:id	Görevi siler ve silinen kaydı yanıtta döndürür	deleteTaskHandler	200 404
GÖREVLER — ARAMA VE LİSTELEME					İSTEĞE BAĞLI
6	GET	/tasks/search?keyword=	Başlık veya açıklamada büyük/küçük harf duyarsız arama	searchTasks	200 400
7	GET	/tasks/assignee/:name	Belirli bir çalışana atanmış görevler (harf duyarsız eşleşme)	getTasksByAssignee	200
RAPORLAMA					İSTEĞE BAĞLI
8	GET	/reports/completed	Durumu `completed` olan görevler ve adet	getCompletedReport	200
9	GET	/reports/pending	Durumu `pending` olan görevler ve adet	getPendingReport	200
10	GET	/reports/summary	Toplam, duruma ve önceliğe göre dağılım	getSummaryReport	200

Tüm yollar ek olarak global `404` (tanımsız yol), `400` (bozuk JSON gövdesi) ve `500` (beklenmeyen hata) yanıtlarını üretebilir.

GET /tasks sorgu parametreleri

PARAMETRE	ÖRNEK	DAVRANIŞ	İŞLEM SIRASI
status	?status=completed	Durumu birebir eşleşen görevler	1 · filtre
priority	?priority=high	Önceliği birebir eşleşen görevler	1 · filtre
sort	?sort=createdAt · ?sort=-createdAt	Belirtilen alana göre artan; başında `-` varsa azalan sıralama	2 · sıralama
page, limit	?page=1&limit=10	Sayfalama; biri verilirse diğeri varsayılan alır (`page=1` , `limit=10`). Yanıta `page` , `limit` , `count` eklenir	3 · sayfalama

Parametreler birlikte kullanılabilir. Örneğin `GET /tasks?status=pending&sort=-createdAt&page=1&limit=5` önce filtreler, ardından en yeniden eskiye sıralar ve ilk 5 kaydı döndürür; `total` sayfalama öncesi eşleşen kayıt sayısını gösterir.

ZİNCİR · KAPSAM · GÖREV

07 Middleware Envanteri

SIRA	MİDDLEWARE	KAPSAM	GÖREV	STATÜ
1	**express.json()**	Global	JSON istek gövdesini ayrıştırıp `req.body` 'ye yerleştirir; bozuk JSON'da hatayı zincire iletir	YERLEŞİK
2	**logger**	Global	`[ISO zaman] METHOD URL` satırını konsola yazar ve `logs/requests.log` dosyasına asenkron ekler; ardından `next()`	ZORUNLU
3	**validateTaskCreate**	POST /tasks	Dört zorunlu alanı denetler; hata varsa `400` ile zinciri sonlandırır	İSTEĞE BAĞLI
3	**validateTaskUpdate**	PUT /tasks/:id	Yalnızca gönderilen alanları denetler; `status` enum kontrolü dahil	İSTEĞE BAĞLI
4	**notFound**	Router'lardan sonra	Hiçbir route ile eşleşmeyen istekleri `404` ile yanıtlar	EK
5	**errorHandler**	Zincirin sonu	Dört parametreli hata middleware'i; hatayı loglar, `err.status || 500` ile JSON döner	EK

Log formatı `logs/requests.log`

```
[2026-10-18T13:03:31.248Z] GET /
[2026-10-18T13:03:31.270Z] POST /tasks
[2026-10-18T13:03:31.279Z] POST /tasks
[2026-10-18T13:03:31.344Z] PUT /tasks/a306f678-…
```

Zincir tanımı `src/app.js`

```
app.use(express.json());
app.use(logger);
app.use('/tasks', taskRoutes);
app.use('/reports', reportRoutes);
app.use(notFound);
app.use(errorHandler);
```

HTTP DURUM KODLARI · HATA SÖZLEŞMESİ

08 Hata Yönetimi ve Durum Kodları

KOD	DURUM	NE ZAMAN DÖNER?	ÜRETEN
200	OK	Başarılı okuma / güncelleme / silme	GET, PUT ve DELETE işlemleri başarıyla tamamlandığında	`controller'lar`
201	Created	Kaynak oluşturuldu	Yeni görev kaydedildiğinde	`createTaskHandler`
400	Bad Request	Geçersiz veya eksik veri	Doğrulama ihlali, `keyword` parametresi eksik, bozuk JSON gövdesi	`validateTask*, searchTasks, errorHandler`
404	Not Found	Kaynak / yol bulunamadı	Verilen `id` ile görev yok ya da tanımsız bir endpoint çağrıldı	`controller'lar, notFound`
500	Server Error	Beklenmeyen sunucu hatası	Yakalanmamış bir istisna oluştuğunda	`errorHandler`

400 doğrulama

```
{
  "success": false,
  "message": "Gecersiz veya eksik veri.",
  "errors": [
    "status su degerlerden biri olmalidir: pending, in-progress, completed."
  ]
}
```

404 kayıt yok

```
{
  "success": false,
  "message": "ID'si yok-123 olan gorev bulunamadi."
}
```

404 tanımsız yol

```
{
  "success": false,
  "message": "Endpoint bulunamadi: /gecersiz-yol"
}
```

GERÇEK ÇALIŞTIRMADAN ALINMIŞ ÇIKTILAR

09 Örnek İstek ve Yanıtlar

Aşağıdaki yanıtlar, API'nin boş bir veri deposuyla yerel olarak çalıştırılması sırasında alınmıştır (18 Ekim 2026).

POST /tasks — **201 Created**

```
{
  "title": "Backend API tasarla",
  "description": "TaskFlow REST API endpointlerini
tasarla",
  "priority": "high",
  "assignee": "musa"
}
```

Yanıt

```
{
  "success": true,
  "data": {
    "id": "385a8373-d33e-4fd8-9c01-805adf67a2fd",
    "title": "Backend API tasarla",
    "status": "pending",
    "priority": "high",
    "assignee": "musa",
    "createdAt": "2026-10-18T13:03:31.271Z", …
  }
}
```

POST /tasks — **400 Bad Request**

```
{ "title": "" }
```

```
{
  "success": false,
  "message": "Gecersiz veya eksik veri.",
  "errors": [
    "title alani zorunludur ve bos olamaz.",
    "description alani zorunludur ve bos olamaz.",
    "priority alani zorunludur ve su degerlerden biri
olmalidir: low, medium, high.",
    "assignee alani zorunludur ve bos olamaz."
  ]
}
```

PUT /tasks/a306f678-… — **200 OK**

```
{ "status": "completed" }
```

```
{
  "success": true,
  "data": {
    "id": "a306f678-0c65-4f31-837f-591c7ffcf29b",
    "title": "Postman koleksiyonu hazirla",
    "status": "completed",
    "priority": "low",
    "createdAt": "2026-10-18T13:03:31.287Z",
    "updatedAt": "2026-10-18T13:03:31.344Z", …
  }
}
```

GET /tasks?page=1&limit=2&sort=-createdAt

```
{
  "success": true,
  "total": 3,
  "page": 1,
  "limit": 2,
  "count": 2,
  "data": [
    { "title": "Postman koleksiyonu hazirla", … },
    { "title": "Veri modelini dokumante et", … }
  ]
}
```

GET /reports/summary — **200 OK**

```
{
  "success": true,
  "data": {
    "total": 3,
    "pending": 2,
    "inProgress": 0,
    "completed": 1,
    "byPriority": { "low": 1, "medium": 1, "high": 1 }
  }
}
```

GET /tasks/search — **400**

```
{
  "success": false,
  "message": "keyword query parametresi zorunludur."
}
```

DELETE /tasks/a306f678-… — **200 OK**

```
{
  "success": true,
  "message": "Gorev silindi.",
  "data": { "id": "a306f678-0c65-4f31-837f-591c7ffcf29b",
… }
}
```

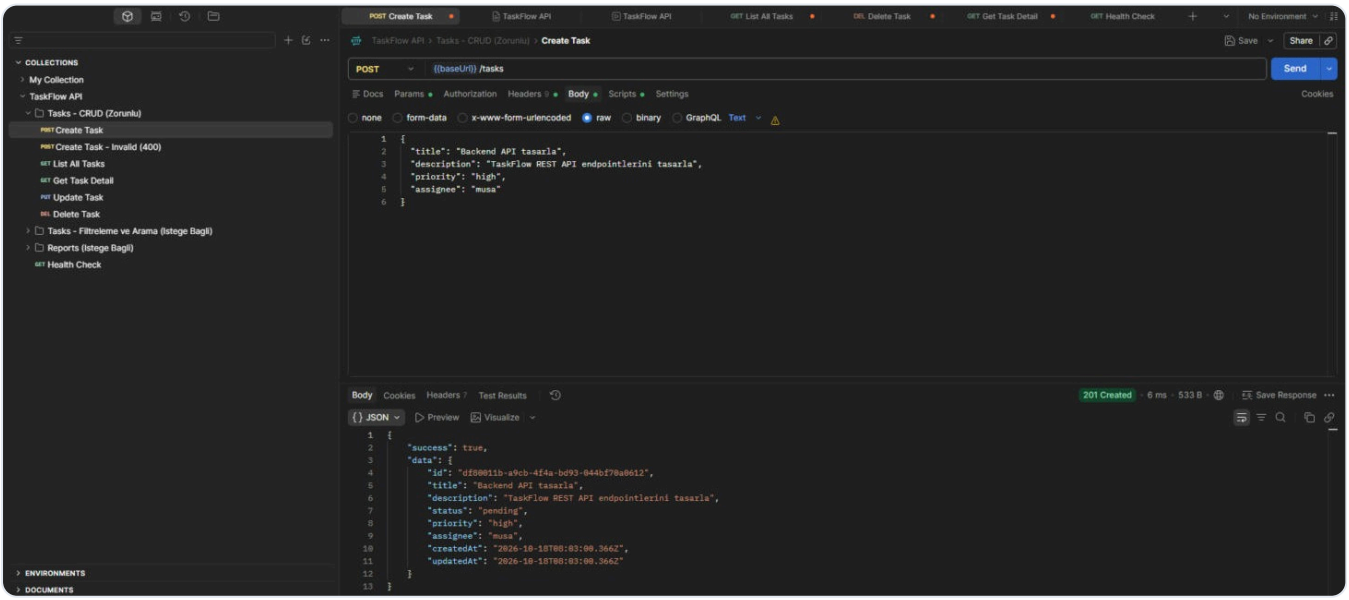
POSTMAN KOLEKSİYONU · TEST KANITLARI

10 Test Envanteri

`postman/TaskFlow.postman_collection.json` koleksiyonu tüm endpointleri kapsayan **16 hazır istek** içerir. `Create Task` isteğindeki test betiği, dönen görev kimliğini `taskId` koleksiyon değişkenine yazar; detay, güncelleme ve silme istekleri bu değişkeni otomatik kullanır.

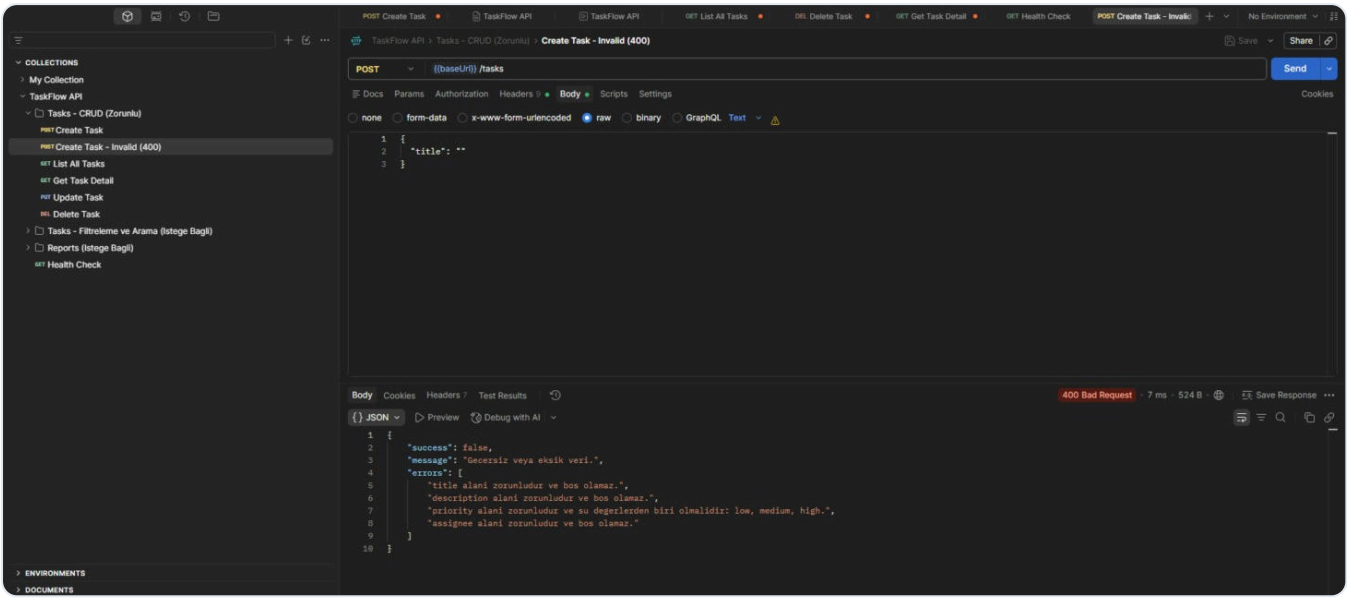
KLASÖR	İSTEK	METHOD	URL
(kök)	Health Check	GET	{{baseUrl}}/
Tasks — CRUD ZORUNLU	Create Task TEST BETİĞİ	POST	{{baseUrl}}/tasks
	Create Task — Invalid (400)	POST	{{baseUrl}}/tasks
	List All Tasks	GET	{{baseUrl}}/tasks
	Get Task Detail	GET	{{baseUrl}}/tasks/{{taskId}}
	Update Task	PUT	{{baseUrl}}/tasks/{{taskId}}
	Delete Task	DELETE	{{baseUrl}}/tasks/{{taskId}}
Tasks — Filtreleme ve Arama İSTEĞE BAĞLI	Filter by Status	GET	/tasks?status=completed
	Filter by Priority	GET	/tasks?priority=high
	Search by Keyword	GET	/tasks/search?keyword=backend
	List by Assignee	GET	/tasks/assignee/musa
	Pagination	GET	/tasks?page=1&limit=10
	Sort by createdAt	GET	/tasks?sort=createdAt
Reports İSTEĞE BAĞLI	Completed Tasks Report	GET	/reports/completed
	Pending Tasks Report	GET	/reports/pending
	Summary Report	GET	/reports/summary

Test kanıtı 1 — başarılı oluşturma



Şekil 1. POST /tasks — geçerli gövde ile **201 Created**; sunucu `id`, `status: "pending"`, `createdAt` ve `updatedAt` alanlarını otomatik üretir.

Test kanıtı 2 — geçersiz veri



Şekil 2. `POST` /tasks — `{ "title": "" }` gövdesi ile **400 Bad Request**; dört alan için ayrı ayrı açıklayıcı hata mesajı döner.

İSTER → BİLEŞEN EŞLEŞMESİ

11 Gereksinim Karşılama Matrisi

GEREKSİNİM	TÜR	KARŞILAYAN BİLEŞEN	DURUM
Görev oluşturma	ZORUNLU	`POST /tasks · createTaskHandler`	✓
Görev listeleme	ZORUNLU	`GET /tasks · getAllTasks`	✓
Görev detayı	ZORUNLU	`GET /tasks/:id · getTaskById`	✓
Görev güncelleme	ZORUNLU	`PUT /tasks/:id · updateTaskHandler`	✓
Görev silme	ZORUNLU	`DELETE /tasks/:id · deleteTaskHandler`	✓
Logger middleware (method, endpoint, zaman)	ZORUNLU	`middleware/logger.js → konsol + logs/requests.log`	✓
Veri doğrulama	İSTEĞE BAĞLI	`middleware/validateTask.js`	✓
Durum / öncelik filtreleme	İSTEĞE BAĞLI	`GET /tasks?status= · ?priority=`	✓
Anahtar kelime arama	İSTEĞE BAĞLI	`GET /tasks/search?keyword=`	✓
Kişiye göre listeleme	İSTEĞE BAĞLI	`GET /tasks/assignee/:name`	✓
Sayfalama ve sıralama	İSTEĞE BAĞLI	`GET /tasks?page=&limit= · ?sort=`	✓
Raporlama	İSTEĞE BAĞLI	`/reports/completed · /pending · /summary`	✓
Dokümantasyon	TESLİM	`README.md · docs/ (4 doküman)`	✓
Postman ile test	TESLİM	`postman/ koleksiyonu · docs/screenshots/`	✓

KURULUM · KOMUTLAR · YAPILANDIRMA

12 Kurulum ve Çalıştırma

1 Depoyu klonlayın

```
$ git clone <repo-url>
$ cd TNC_Backend
```

2 Bağımlılıkları yükleyin

```
$ npm install
```

3 (Opsiyonel) Ortam dosyası

```
$ cp .env.example .env   # PORT=3000
```

4 Sunucuyu başlatın

```
$ npm start
TaskFlow API http://localhost:3000 adresinde
calisiyor.
```

KOMUT	AÇIKLAMA
`npm start`	`node src/server.js` — üretim modu
`npm run dev`	`nodemon src/server.js` — otomatik yeniden başlatma

DEĞİŞKEN	VARSAYILAN
PORT	`3000` — `.env` yoksa kullanılır

Doğrulama: Sunucu çalışırken `GET http://localhost:3000/` isteği `"TaskFlow API calisiyor."` mesajını döndürmelidir. Ardından Postman koleksiyonu import edilerek istekler sırasıyla çalıştırılabilir.

Git'e dahil edilmeyenler

`node_modules/` · `.env` · `logs/*.log` · `npm-debug.log*`

V1.0.0 SONRASI YOL HARİTASI

13 Gelecek Sürüm Planı

Kalıcı veritabanı — YÜKSEK

Dosya tabanlı depo yerine MongoDB veya PostgreSQL; eşzamanlı yazmalarda tutarlılık ve büyük veri setlerinde performans.

Kimlik doğrulama — YÜKSEK

JWT tabanlı oturum, rol bazlı yetkilendirme; görevlerin yalnızca yetkili kullanıcılarca güncellenmesi.

Şema doğrulamasının sıkılaştırılması — ORTA

Oluşturmada `status` için enum kontrolü, güncellemede izin verilen alan listesi ve `sort` alanı için beyaz liste.

Otomatik testler — ORTA

Jest + Supertest ile endpoint bazlı entegrasyon testleri ve CI üzerinde otomatik çalıştırma.

OpenAPI / Swagger — DÜŞÜK

API sözleşmesinin makine tarafından okunabilir şekilde yayımlanması ve etkileşimli doküman arayüzü.

Proje varlığı — DÜŞÜK

Görevleri projelere bağlayan `Project` varlığı ve proje bazlı raporlar.

İbrahim Gürkan Kaya
Node.js ile Backend Programlama — Bitirme Projesi

TaskFlow API · v1.0.0
Ekim 2026